```
print("Value is", 42)
```

Because print is now a function, you can pass it as an argument, use keyword arguments like *sep* and *end*, or wrap it:

```
def shout(msg):
    print(msg.upper())

shout("hello")
```

True division vs floor division (/ vs //): In Python 2, the / operator did integer division when both operands were integers, and floating-point division otherwise. This tended to produce insidious bugs, particularly when programmers expected decimal results.

```
#Python 2
print 5 / 2     # Outputs: 2 (integer division)
print 5 / 2.0   # Outputs: 2.5 (float division)

#Python 3
print(5 / 2)    # Outputs: 2.5 (true division)
print(5 // 2)   # Outputs: 2 (floor division)
```

This change enforces mathematical accuracy and avoids unexpected results in calculations, especially in applications like finance, data analysis, and simulations.

Unicode string support by default: Working with text under Python 2 involved switching back and forth between byte strings (str) and Unicode strings (unicode). Blending the two led to difficult-to-debug encoding issues, especially in web and cross-national applications.

```
#Python 2
s = u"こんにちは"  # Unicode string
b = "hello"         # Byte string
print s, b
```

In Python 3, all strings are Unicode by default, and a separate byte type is used for binary data. This simplifies string handling and makes the language more robust for internationalisation.

```
#Python 3
s = "こんにちは" # Unicode by default
b = b"hello"       # Byte string

print(s, b.decode())  # Convert bytes to string for display
```

The result is safer, cleaner code that handles multilingual and encoded data more gracefully, which is critical for APIs, user interfaces, and file processing.

Code examples: Python 2 vs Python 3: Suppose you're coding a script that gathers user feedback, computes an average rating, and prints a 'thank-you' message along with some internationalised text.

Let's begin by seeing how this would look in Python 2, then how Python 3 refines it with improved syntax, string manipulation, and safer behaviour.

```
#Python 2
feedback = [
    {"name": "Alice", "rating": 5, "comment": u"Great
service!"},
    {"name": "Bob", "rating": 4, "comment": u"Buen trabajo"},
    {"name": "Chika", "rating": 3, "comment": u"良いサービス"}
]

total = 0
for entry in feedback:
    print "User:", entry["name"]
    print "Comment:", entry["comment"].encode("utf-8")
    total += entry["rating"]

average = total / len(feedback)  # Integer division!
print "Average rating:", average
```

The issues in Python 2 are:

- You need to encode Unicode strings manually (.encode("utf-8")).
- Integer division (/) silently discards the decimal.
- *print* is a statement, limiting composability and formatting.

```
#Python 3
feedback = [
    {"name": "Alice", "rating": 5, "comment": "Great
service!"},
    {"name": "Bob", "rating": 4, "comment": "Buen trabajo"},
  {"name": "Chika", "rating": 3, "comment": "良いサービス"}
]

total = 0
for entry in feedback:
    print(f"User: {entry['name']}")
    print(f"Comment: {entry['comment']}")
    total += entry["rating"]

average = total / len(feedback)  # True division!
print(f"Average rating: {average:.1f}")
```

This Python 3 script is easier to read, safer (particularly with Unicode and division), and simpler to expand to — for